UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026).
   Answers are unofficial reference answers, not official UTM marking schemes.

2. Marking points show the expected answer and mark allocation; check them against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Approved real questions only; no generators, placeholders or parametric variants.
   Questions and answers are for local study use only.

5. Structure derived from the supplied CSC429 papers; every question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 11 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION 1 (5 marks)

Source: CSC429-2019-06, page [6]

Unofficial model answer:

a) A tightly coupled multiprocessor system shares a single global main memory (and typically a common clock and OS image) that all processors access directly, so inter-processor communication is via shared memory with low latency; a loosely coupled multiprocessor system gives each processor its own private local memory and the nodes communicate by message passing over an interconnection network (e.g., LAN), so there is no shared global memory and communication latency is higher. (Any one clear contrast, e.g., shared memory vs. private memory/message passing, earns the mark.)
b) The three types of cluster architecture are: (1) shared-disk cluster (all nodes share a common disk subsystem, e.g., a SAN, but have private memory); (2) shared-nothing cluster (each node has its own processor, memory and disk, and nodes cooperate only by message passing); and (3) shared-memory cluster (nodes are connected to a common shared memory or use a DSM/software shared-memory layer).

Marking points:

• *1 mark (a): Correctly states one major difference, e.g., tightly coupled shares a single global memory while loosely coupled has private memory per node and uses message passing.*

• *1 mark (b): Names shared-disk cluster.*

• *1 mark (b): Names shared-nothing cluster.*

• *1 mark (b): Names shared-memory cluster.*

• *1 mark (b): Brief correct description/characteristic of each of the three cluster types (or equivalent accurate naming with explanation).*

Model confidence: medium.

Model note: The question does not specify a particular textbook taxonomy for cluster architectures; the answer uses the common shared-disk / shared-nothing / shared-memory classification. If the lecturer's notes use a different three-way classification (e.g., high-availability, load-balancing, high-performance clusters), that alternative should be accepted. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 2 (5 marks)

Source: CSC429-2014-12, page [2]

Unofficial model answer:

Von Neumann architecture: The classic stored-program computer model in which a single memory holds both instructions and data, and the CPU fetches instructions and operands over a shared bus. A correct drawing shows five blocks: (1) Input unit, (2) Memory unit, (3) Arithmetic Logic Unit (ALU), (4) Control Unit, and (5) Output unit, with the ALU and Control Unit grouped as the CPU (Central Processing Unit). Arrows should show the flow: input → memory; memory ↔ CPU (address bus, data bus, control bus); CPU → output. The Control Unit fetches and decodes instructions and generates control signals; the ALU performs arithmetic and logic operations; memory stores both program and data; input/output units transfer data to and from the outside world. Example for each component: Input unit — keyboard, mouse, scanner; Memory unit — RAM (main memory), ROM/cache; ALU — performs addition (e.g., ADD R1,R2) or logical AND/OR/comparison; Control unit — the program counter and instruction decoder that sequence fetch-decode-execute; Output unit — monitor/display, printer, speaker. (A labelled block diagram is required in the exam; the text above gives the blocks and connections that must appear.)

Marking points:

• *1 mark: Correct block diagram showing the five units — input, memory, ALU, control unit, output — with CPU*

© Hak Cipta Universiti Teknologi MARA

grouping of ALU + control unit.

• 1 mark: Correct data/control flow arrows (input→memory, memory↔CPU via buses, CPU→output) and shared memory for instructions and data.

• 1 mark: Appropriate example for input and output units (e.g., keyboard/scanner; monitor/printer).

• 1 mark: Appropriate example for memory unit and ALU (e.g., RAM/ROM; addition/logic operation).

• 1 mark: Appropriate example/role for control unit (e.g., PC, instruction decoder, fetch-decode-execute sequencing).

Model confidence: high.

Model note: No figure was supplied; the answer describes the standard Von Neumann block diagram that the student must draw. Marking assumes a 5-mark allocation with roughly one mark per component/example plus diagram correctness; exact mark split may vary.

QUESTION 3 (5 marks)

Source: CSC429-2018-01, page [2]

Unofficial model answer:

a) The personal computer (the Altair 8800, 1975, cited in Figure 1) belongs to the FOURTH computer generation, the microprocessor/VLSI era (roughly 1971 onward). This generation is defined by the invention of the microprocessor — the Intel 4004 (1971) and later 8080 — which put a complete CPU on a single chip and made cheap desktop machines like the Altair possible.

b) One other major discovery of the fourth generation is Very Large Scale Integration (VLSI): packing thousands to millions of transistors onto one silicon chip. VLSI (together with the microprocessor it enabled) produced dramatic reductions in size, cost and power per function, and gave rise to microcomputers, inexpensive semiconductor memory, and the graphical user interface/personal-computing revolution. (Acceptable alternatives: the microprocessor itself if not used in (a), or large-scale semiconductor memory replacing core memory.)

c) Laptop and hand-held devices are feasible now because of continued scaling of VLSI into ULSI and system-on-chip (SoC) design: far more transistors per chip at lower cost, lower power and higher clock speed, plus low-power CMOS, high-density memory, flat-panel displays and wireless/battery technology. Compared with earlier generations (vacuum tubes, then discrete transistors, then SSI/MSI integrated circuits), which were bulky, hot, power-hungry and expensive, modern devices integrate CPU, memory and I/O on a few chips, so the same computing power that once filled a room now fits in a hand-held device.

Marking points:

• 1 mark (a): Correctly identifies the fourth generation (microprocessor/VLSI era, ~1971 onward) as the generation in which the personal computer appeared.

• 2 marks (b): Names one valid major discovery of that generation (e.g., VLSI, or the microprocessor) — 1 mark; gives a correct, relevant discussion of its significance/impact — 1 mark.

• 2 marks (c): Explains feasibility via VLSI/ULSI/SoC scaling and low-power, high-density integration — 1 mark; contrasts with previous generations (tubes/transistors/SSI-MSI: bulky, costly, power-hungry) to justify the advance — 1 mark.

Model confidence: high.

Model note: Figure 1 is only a text snippet (Altair 8800, 1975); no diagram content is needed. Generation numbering follows the standard CSC429 convention (1st: vacuum tubes; 2nd: transistors; 3rd: ICs; 4th: microprocessors/VLSI; 5th: AI/ULSI). This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 4 (5 marks)

Source: CSC429-2018-12, page [2]

Unofficial model answer:

a) A byte is a unit of digital information that consists of 8 bits. It is the smallest addressable unit of memory in

© Hak Cipta Universiti Teknologi MARA

most computer architectures and can represent 256 distinct values (2^8), commonly used to store a single character or a small integer.

b) Base 10 is not suitable for computer number representation because digital computers use binary logic, where each bit has two stable states (0 and 1). Implementing ten distinct voltage levels for decimal digits would be complex, error-prone, and costly. Binary representation maps directly to the two-state nature of electronic components, enabling reliable storage and processing.

c) In IEEE floating point representation, the exponent field uses a biased representation. For single precision, the bias is 127, and the exponent field is 8 bits. The actual exponent value is obtained by subtracting the bias from the stored value. The maximum stored exponent value is 255, but 255 is reserved for special values (infinity and NaN). The maximum normal exponent stored is 254, which gives an actual exponent of 254 - 127 = 127. Thus, the maximum exponent value is 127, not 128, because the value 128 would require a stored exponent of 255, which is reserved.

Marking points:

• 1 mark: Definition of byte as 8 bits (or 8 bits grouped as a unit).

• 1 mark: Explanation that computers use binary (two-state) logic, making base 10 impractical.

• 1 mark: Mention of complexity/cost/reliability issues with ten-state devices.

• 1 mark: Identification of exponent bias (127) and reserved exponent value (255) for special cases.

• 1 mark: Calculation showing max normal exponent = 254 - 127 = 127, and why 128 would require reserved value 255.

Model confidence: high.

Model note: The answer assumes single-precision IEEE 754 format. The question does not specify precision, but the value 127 is specific to single precision. For double precision, the bias is 1023 and max exponent is 1023, so the question implicitly refers to single precision.

QUESTION 5 (10 marks)

Source: CSC429-2018-06, page [5]

Unofficial model answer:

a) Polling I/O problem: In polling, the CPU repeatedly checks the status register of the I/O device in a busy-wait loop until the device is ready. This wastes CPU cycles because the processor cannot perform other useful work while waiting; the CPU is tied up in a tight loop, and the efficiency depends on how often the device becomes ready. If the device is slow, the CPU spends a long time polling, reducing overall system throughput. Also, polling does not support multitasking well because the CPU is occupied.

b) Buffering improves overall device performance by smoothing the speed mismatch between the CPU and I/O devices. A buffer is a temporary storage area (in memory or device controller) that holds data during transfer. It allows the CPU to continue executing other instructions while the I/O device fills or drains the buffer, reducing the number of context switches and interrupts. For example, double buffering allows the CPU to process one buffer while the device fills another, overlapping computation and I/O. Buffering also helps handle bursty I/O traffic and allows devices with different data transfer rates to communicate more efficiently, improving overall throughput and reducing CPU idle time.

c) DMA (Direct Memory Access) improves I/O transfer performance compared to polling and interrupt-driven I/O by offloading the data transfer task from the CPU to a dedicated DMA controller. In polling, the CPU busy-waits; in interrupt I/O, the CPU is interrupted for each data word or byte, causing overhead from context switching and interrupt handling. With DMA, the CPU programs the DMA controller with the source/destination addresses and transfer count, then the DMA controller transfers data directly between the I/O device and memory without CPU involvement. The CPU is only interrupted once at the end of the entire block transfer (or on error). This greatly reduces CPU overhead, allows the CPU to perform other tasks during the transfer, and achieves higher data transfer rates, especially for large blocks of data. Thus DMA improves overall I/O

© Hak Cipta Universiti Teknologi MARA

performance and system efficiency.

Marking points:

• 1 mark: Polling wastes CPU cycles in busy-wait loop.

• 1 mark: CPU cannot do other work while polling; inefficient for slow devices.

• 1 mark: Buffering smooths speed mismatch between CPU and I/O device.

• 1 mark: Buffer allows CPU to continue other work while I/O fills/drains buffer.

• 1 mark: Double buffering overlaps computation and I/O.

• 1 mark: Buffering reduces number of interrupts/context switches.

• 1 mark: DMA offloads data transfer from CPU to DMA controller.

• 1 mark: CPU programs DMA with addresses and count, then DMA transfers directly to/from memory.

• 1 mark: CPU interrupted only once per block, reducing overhead vs per-word interrupts.

• 1 mark: DMA allows CPU to do other work during transfer and achieves higher throughput.

Model confidence: high.

Model note: This is an unofficial model answer; no official marking scheme was provided. The question text contains a typo '1/0' for 'I/O'. The answer assumes standard textbook definitions of polling, buffering, and DMA.

QUESTION 6 (10 marks)

Source: CSC429-2024-01, page [5]

Unofficial model answer:

a) MIMD with distributed memory: The architecture consists of N independent processing nodes. Each node has its own Control Unit (CU), its own Processing Unit (PU), and its own Local Memory (LM). The CUs are not synchronized by a single global clock/instruction stream; each CU fetches and decodes instructions independently for its PU. The LMs are private to each node and are not directly addressable by other nodes. Nodes are connected by an interconnection network (e.g., a mesh, hypercube, or network switch) so that PUs can exchange messages/data explicitly. A suitable drawing shows a row/column of identical nodes, each drawn as a box containing CU, PU, and LM, with the PU connected to its LM and to the interconnection network; the network links the nodes. No single shared memory block is drawn. (Figure limitation: the original question asks for a drawing; this text describes the required diagram because no figure can be rendered in JSON.)

b) MIMD with shared memory: The architecture consists of N Processing Units (PUs), each with its own Control Unit (CU), so each PU executes its own instruction stream independently (multiple instruction streams). All PUs are connected through an interconnection network (e.g., a shared bus or crossbar switch) to one global shared memory. The shared memory is directly addressable by every PU, so communication between processors occurs by writing to and reading from shared memory locations. A suitable drawing shows several CU+PU pairs, each CU feeding its PU, all PUs attached to a common interconnection network, and that network attached to a single shared-memory block. (Figure limitation: same as above; the diagram is described rather than drawn.)

c) Virus attack on memory: i) In MIMD with distributed memory, a virus that infects one node's Local Memory can corrupt only that node's local data and programs (and possibly the node's CU/PU if the virus is executed). Because each LM is private and there is no globally shared address space, the infection does not directly spread through memory to other nodes. However, the infected node can send corrupted messages or infected files over the interconnection network to other nodes, so the virus may spread indirectly through communication; the blast radius is initially limited to one node but can grow via message passing. ii) In MIMD with shared memory, a virus that attacks the shared memory can corrupt data or code in the single global address space that all PUs access. Because every PU reads and writes the same shared memory, the corruption is immediately visible to all processors and can affect all running programs; the virus can spread rapidly across the whole system through shared memory locations. Recovery is harder because there is no

© Hak Cipta Universiti Teknologi MARA

isolated private copy per node, and one infected PU can write malicious code into shared memory for other PUs to execute.

d) One example of MIMD with distributed memory: a cluster of networked workstation PCs (or a Beowulf cluster) in which each computer has its own CPU, local RAM, and operating system, and the computers communicate by message passing over a network. (Other acceptable examples: a distributed-memory multicomputer such as Intel Paragon, IBM SP2, or a modern HPC cluster with InfiniBand.)

Marking points:

• a) 3 marks: 1 mark for showing multiple nodes each with its own CU, PU, and LM; 1 mark for correct CU-PU-LM connections within each node; 1 mark for an interconnection network linking the nodes and no shared global memory.

• b) 3 marks: 1 mark for multiple CU+PU pairs (multiple instruction streams); 1 mark for all PUs connected to a common interconnection network; 1 mark for a single shared memory block accessible by all PUs.

• c) i) 1.5 marks: 1 mark for stating that only the infected node's local memory is directly corrupted / limited direct spread; 0.5 mark for noting possible indirect spread via messages over the network.

• c) ii) 1.5 marks: 1 mark for stating that shared memory corruption affects all PUs / the whole system; 0.5 mark for noting rapid spread and difficulty of isolation because all processors share the same address space.

• d) 1 mark: one correct example of a distributed-memory MIMD system (e.g., cluster of workstations/PCs with message passing, Beowulf cluster, or a named distributed-memory multicomputer).

Model confidence: high.

Model note: The question asks for drawings in parts (a) and (b); because the response format is JSON text, the diagrams are described in words rather than rendered. The marking allocation is an unofficial suggested split of the 10 marks (3+3+3+1) and is not an official UTM scheme. Part (c) is answered conceptually because the exact virus behaviour is not specified in the question.

QUESTION 7 (10 marks)

Source: CSC429-2018-12, page [3]

Unofficial model answer:

a) Sign and magnitude: Convert 97_{10} to 8-bit sign-magnitude: 97 = 01100001_2 (sign 0, magnitude 1100001). FACEH is a 16-bit hexadecimal value; to add it to an 8-bit number, use 16-bit sign-magnitude representation. 97_{10} = 0000 0000 0110 0001_2. FACEH = 1111 1010 1100 1110_2. In sign-magnitude, the sign bit is the MSB and the remaining bits are magnitude. Since the signs differ (0 vs 1), subtract the smaller magnitude from the larger magnitude and keep the sign of the larger. Magnitudes: 0000 0000 0110 0001_2 = 97_{10}; 111 1010 1100 1110_2 = 64206_{10}. Larger magnitude is 64206 (negative number). 64206 − 97 = 64109_{10} = 1111 1010 1000 1101_2. Result sign is negative (1), so sign-magnitude result = 1111 1010 1000 1101_2 = FA8DH. Check: 97 + (−64206) = −64109, and FA8DH in sign-magnitude is −(7A8DH) = −(0x7A8D) = −31373? Wait, 0x7A8D = 31373, not 64109. Correction: magnitude bits are 111 1010 1000 1101_2 = 0x7A8D = 31373, so the value is −31373, which is wrong. Let's recompute: FACEH as 16-bit sign-magnitude: sign bit = 1, magnitude = 111 1010 1100 1110_2 = 0x7ACE = 31438_{10}. So FACEH represents −31438, not −64206. Then 97 + (−31438) = −31341. Magnitude subtraction: 31438 − 97 = 31341 = 0x7A6D = 0111 1010 0110 1101_2. With sign bit 1: 1111 1010 0110 1101_2 = FA6DH. So sign-magnitude result = FA6DH. Work flow: (1) represent both in 16-bit sign-magnitude; (2) compare signs; (3) since signs differ, subtract smaller magnitude from larger; (4) attach sign of larger magnitude. Result: FA6DH (i.e., -31341_{10}). b) 1's and 2's complement: Use 16-bit two's complement. 97_{10} = 0000 0000 0110 0001_2. FACEH is already a 16-bit pattern; in two's complement it represents -1330_{10} because FACEH = 1111 1010 1100 1110_2, and its two's complement value = −(0000 0101 0011 0010_2) = -1330_{10}. Addition: 0000 0000 0110 0001 + 1111 1010 1100 1110 = 1111 1011 0010 1111_2 = FB2FH. No carry out, so no overflow. In 1's complement, FACEH represents −(0000 0101 0011 0001_2) = -1329_{10}. 1's complement addition: 0000 0000 0110 0001 + 1111 1010 1100 1110 = 1111 1011 0010 1111_2 = FB2FH. There is no end-around carry, so result remains FB2FH. To interpret as 1's complement: sign bit 1, magnitude = complement of lower 15 bits = 0000 0100 1101 0000_2 = 1232_{10}, so value = -1232_{10}. Check: 97 + (−1329) =

© Hak Cipta Universiti Teknologi MARA

−1232. Work flow: convert to 16-bit; add; for 1's complement, if carry out add it to LSB (none here); for 2's complement, discard carry out (none here). Results: 1's complement FB2FH (-1232_{10}); 2's complement FB2FH (-1330_{10}).

Marking points:

• 1 mark: correct 8-bit/16-bit sign-magnitude representation of 97 (0000 0000 0110 0001)

• 1 mark: correct interpretation of FACEH in sign-magnitude (sign=1, magnitude=7ACEH)

• 1 mark: correct sign-magnitude addition workflow (compare signs, subtract magnitudes, attach sign)

• 1 mark: correct sign-magnitude result FA6DH (or equivalent)

• 1 mark: correct 16-bit two's complement representation of 97 and FACEH

• 1 mark: correct two's complement addition showing sum FB2FH

• 1 mark: correct handling of carry/overflow in two's complement (no carry out, no overflow)

• 1 mark: correct 1's complement addition showing sum FB2FH

• 1 mark: correct handling of end-around carry in 1's complement (none needed)

• 1 mark: correct interpretation of final 1's complement and 2's complement results (FB2FH = −1232 and −1330 respectively)

Model confidence: medium.

Model note: Assumes 16-bit word length because FACEH is a 16-bit hexadecimal value; 97 is extended to 16 bits. The question does not specify word size, so 8-bit representation would be insufficient for FACEH. Sign-magnitude interpretation of FACEH as a negative number is assumed; if FACEH is treated as an unsigned bit pattern, the sign-magnitude result differs. The final 1's complement and 2's complement bit patterns are identical (FB2FH) but represent different decimal values due to different encodings.

QUESTION 8 (10 marks)

Source: CSC429-2014-06, page [4]

Unofficial model answer:

a) Interrupt-driven I/O is an I/O technique in which the processor initiates an I/O operation and then continues executing other instructions; the I/O module, upon completing the transfer or requiring service, sends an interrupt request to the processor. The processor suspends its current program, saves the context, and transfers control to an interrupt service routine (ISR) that services the I/O device. This avoids the processor having to repeatedly poll the device status as in simple programmed I/O.

b) When the processor, main memory and I/O modules share a common bus, two modes of addressing are: (i) Memory-mapped I/O: I/O device registers are assigned addresses in the same address space as main memory, so the processor uses ordinary memory-reference instructions and the same address bus to access I/O; no special I/O instructions are needed, but a portion of the memory address space is consumed. (ii) Isolated (port-mapped) I/O: I/O device registers occupy a separate I/O address space, and the processor uses dedicated I/O instructions (e.g., IN/OUT) with a distinct control signal to distinguish I/O accesses from memory accesses; this preserves the full memory address space but requires special instructions and control lines.

c) Advantage: The processor does not waste time polling; it can execute other programs while I/O proceeds, so CPU utilization and system throughput are improved, especially for slow devices. Disadvantage: Interrupt handling incurs overhead (context save/restore, ISR execution, and possible bus arbitration), and frequent interrupts from many devices can degrade performance; also, interrupt-driven I/O requires more complex hardware and software than programmed I/O.

Marking points:

• 1 mark: Correct definition of interrupt-driven I/O as processor-initiated I/O with device-generated interrupt on completion/need, avoiding polling.

• 1 mark: Mention of context save/restore and ISR transfer.

• 2 marks: Memory-mapped I/O mode correctly described (shared address space, ordinary memory instructions).

• 2 marks: Isolated/port-mapped I/O mode correctly described (separate I/O address space, special I/O

© Hak Cipta Universiti Teknologi MARA

instructions/control signals).

• 1 mark: Advantage: improved CPU utilization/throughput by eliminating polling.

• 1 mark: Disadvantage: interrupt overhead/complexity or performance degradation from frequent interrupts.

• 2 marks: Overall clarity, correct terminology, and completeness (up to 2 marks for coherent explanation across subparts).

Model confidence: high.

Model note: No figure or table was supplied; answer is based on standard computer organization concepts. Mark allocation is an unofficial interpretation of the 10-mark total.

QUESTION 9 (10 marks)

Source: CSC429-2025-02, page [5]

Unofficial model answer:

a) SISD diagram (described, since a figure cannot be drawn in JSON): Draw one MU block at the top, one PU block below it, and one CU block to the left of the PU. A single arrow labelled IS runs from MU into CU (the instruction stream), and a single arrow labelled DS runs from MU into PU (the data stream). CU sends a control arrow to PU. Only one instruction stream and one data stream exist, and instructions are executed sequentially — this is the classic von Neumann uniprocessor. (Limitation: the original figure cannot be reproduced here; the description gives the required blocks and labelled streams.)

b) Parallel architecture: A parallel architecture is a computer organization in which multiple processing elements (PUs/cores) cooperate to execute a program, so that several operations are performed simultaneously rather than one at a time. It is characterised by more than one instruction stream and/or more than one data stream (as in SIMD, MISD and MIMD of Flynn's taxonomy), and by hardware for communication/synchronisation between the processing elements (shared memory or an interconnection network). Parallelism may be at instruction level (pipelining, superscalar) or at processor level (multicore, multiprocessor, cluster). The aim is higher throughput and reduced execution time by exploiting concurrency.

c) One benefit of parallel architecture: Increased performance/throughput — because work is divided among several processing elements, a task can be completed in less time than on a single processor (speedup), and the system can handle larger or more numerous problems. (Other acceptable benefits: better scalability, improved reliability/fault tolerance, better price-performance using many commodity processors.)

d) Loosely-coupled vs strongly-coupled MIMD: In a loosely-coupled (distributed-memory) MIMD system each processor has its own private local memory and its own copy of the operating system; processors communicate by message passing over a network, so memory access to another node is slow and there is no shared global address space. Example: a cluster of workstations or a network of autonomous computers. In a strongly-coupled (shared-memory) MIMD system all processors share a single global main memory and typically one operating system; processors communicate by reading/writing shared variables, so communication is fast but requires synchronisation (locks, semaphores) to avoid race conditions. Example: a symmetric multiprocessor (SMP) multicore server with a common RAM.

Marking points:

• a) 3 marks: 1 mark for correct MU, PU and CU blocks; 1 mark for a single IS arrow from MU to CU; 1 mark for a single DS arrow from MU to PU (sequential single stream implied).

• b) 3 marks: 1 mark for definition of simultaneous execution by multiple processing elements; 1 mark for reference to multiple instruction/data streams (Flynn) or levels of parallelism; 1 mark for communication/synchronisation or the goal of higher throughput.

• c) 2 marks: 1 mark for naming a valid benefit (e.g. speedup/throughput); 1 mark for a correct explanation of why it arises.

• d) 2 marks: 1 mark for the contrast (private memory + message passing vs shared memory + shared variables/one OS); 1 mark for a correct example of each (cluster/workstation network vs SMP/multicore shared-memory system).

Model confidence: high.

© Hak Cipta Universiti Teknologi MARA

Model note: The SISD figure is described in words because a diagram cannot be rendered in JSON; the original figure was not supplied, so the layout follows the standard textbook SISD block diagram. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 10 (10 marks)

Source: CSC429-2017-07, page [4]

Unofficial model answer:

a) A replacement algorithm is needed in a cache because the cache is much smaller than main memory, so when a cache miss occurs and the requested block must be brought in, the cache may already be full. The replacement algorithm decides which existing cache block (line) to evict to make room for the new block. A good replacement policy aims to evict blocks that are least likely to be used again, thereby maximising the hit ratio and minimising unnecessary memory traffic. Without a replacement algorithm, the cache could not handle capacity or conflict misses in a set-associative or fully associative organisation.

b) The main replacement algorithms are: (1) Random replacement — a randomly chosen block is evicted; simple to implement in hardware but gives no regard to usage history. (2) FIFO (First-In, First-Out) — the block that has been in the cache the longest is replaced; easy to implement with a queue or counter, but may evict a frequently used block. (3) LRU (Least Recently Used) — the block that has not been referenced for the longest time is replaced; exploits temporal locality and generally gives good performance, but true LRU is costly to implement for high associativity because it requires tracking the order of all accesses. (4) LFU (Least Frequently Used) — the block with the fewest references is replaced; favours frequently used blocks but can retain stale blocks that were heavily used in the past. (5) Optimal (MIN) replacement — replaces the block that will not be used for the longest time in the future; it is a theoretical benchmark that cannot be implemented in practice because future references are unknown. Other variants include pseudo-LRU (approximates LRU with less hardware) and clock/second-chance algorithms, which are practical approximations of LRU.

Marking points:

- *1 mark: state that cache is smaller than main memory and replacement is needed on a miss when the target set/line is full*
- *1 mark: explain that the algorithm selects a victim block to evict to make space for the new block*
- *1 mark: mention goal of maximising hit ratio / minimising unnecessary evictions of useful blocks*
- *1 mark: name Random replacement and briefly describe it*
- *1 mark: name FIFO and briefly describe it*
- *1 mark: name LRU and briefly describe it*
- *1 mark: name LFU and briefly describe it*
- *1 mark: name Optimal (MIN) and note it is theoretical / not implementable*
- *1 mark: mention at least one practical approximation such as pseudo-LRU or clock/second-chance*
- *1 mark: overall clarity, correct terminology, and coverage of all types*

Model confidence: high.

Model note: The question text does not specify a particular cache organisation; the answer assumes set-associative or fully associative caches where replacement is relevant. The list of replacement types is based on standard computer-organization textbooks; some courses may emphasise a subset (e.g., Random, FIFO, LRU, Optimal) and treat LFU as optional. This is an unofficial model answer, not an official UTM marking scheme.

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION 11 (20 marks)

Source: CSC429-2025-02, page [6]

Unofficial model answer:

The equation in Figure 2 is interpreted as y = A - (2C * D) / (B + A^2). Registers A, B, C, D are fixed; x, y, z are temporary; final result must be in y. Table 1 provides one-address (LOADp, STORp, ADDp, SUBp, MULp, DIVp), two-address (MOV(p,q), ADD(p,q), SUB(p,q), MUL(p,q), DIV(p,q)), and three-address (ADD(p,q,r), SUB(p,q,r), MUL(p,q,r), DIV(p,q,r)) instructions. In two- and three-address notations, the first operand is the destination. For one-address, the accumulator is implicit; we use x as accumulator and y as final storage.

One-address program: LOAD A; MUL A; STOR z; LOAD B; ADD z; STOR z; LOAD C; MUL C; MUL D; STOR x; LOAD A; SUB x; STOR y; LOAD x; DIV z; STOR y. Explanation: compute A^2 into z, then B+A^2 into z; compute 2C*D into x; compute A - x into y; finally divide by z and store in y. This uses only the listed one-address instructions and temporary registers x, y, z.

Two-address program: MOV(x, A); MUL(x, A); ADD(x, B); MOV(z, C); MUL(z, C); MUL(z, D); MOV(y, A); SUB(y, z); DIV(y, x). Explanation: x = A^2 + B; z = 2C*D; y = A - z; y = y / x. The final result is in y. All instructions are from the two-address column.

Three-address program: MUL(x, A, A); ADD(x, x, B); MUL(z, C, C); MUL(z, z, D); SUB(y, A, z); DIV(y, y, x). Explanation: x = A^2 + B; z = 2C*D; y = A - z; y = y / x. The final result is in y. All instructions are from the three-address column.

Note: The figure was supplied only as an image asset; the equation is inferred from the extracted text 'y = A - 2C * D / (B + A2)'. If the intended grouping differs (e.g., (2C*D)/(B+A^2) versus 2C*(D/(B+A^2))), the instruction sequences would change accordingly. The answer assumes standard operator precedence and left-to-right evaluation for multiplication/division.

Marking points:

*• 1 mark: Correctly interpret the equation as y = A - (2C * D) / (B + A^2) or equivalent, showing understanding of operator precedence.*

• 1 mark: State that one-address instructions use an implicit accumulator and that x, y, z are used as temporary/final storage.

• 2 marks: One-address program correctly computes A^2 and B+A^2 using LOAD, MUL, ADD, STOR (partial credit for minor sequencing errors).

*• 2 marks: One-address program correctly computes 2C*D using LOAD, MUL, MUL, STOR.*

*• 2 marks: One-address program correctly computes A - (2C*D) and divides by (B+A^2), storing final result in y.*

• 1 mark: Two-address program correctly computes x = A^2 + B using MOV, MUL, ADD.

*• 1 mark: Two-address program correctly computes z = 2C*D using MOV, MUL, MUL.*

• 2 marks: Two-address program correctly computes y = A - z and y = y / x, final result in y.

• 1 mark: Three-address program correctly computes x = A^2 + B using MUL, ADD.

*• 1 mark: Three-address program correctly computes z = 2C*D using MUL, MUL.*

• 2 marks: Three-address program correctly computes y = A - z and y = y / x, final result in y.

• 1 mark: All instructions used are from Table 1 and correct assembly notation (operand order, register names) is followed.

• 1 mark: Overall clarity, correct use of temporary registers, and final result stored in y for all three programs.

• 1 mark: Acknowledges any ambiguity in the figure/equation (e.g., grouping) and states assumptions.

Model confidence: medium.

*Model note: The equation was inferred from the extracted text 'y = A - 2C * D / (B + A2)' because the figure was provided only as an image asset without a machine-readable equation. The answer assumes standard precedence: multiplication and division before subtraction, and left-to-right for * and /. If the intended equation is different (e.g., y = A - 2C * (D/(B+A^2))), the programs would*

© Hak Cipta Universiti Teknologi MARA

need adjustment. Table 1 was provided as a native table and is used as-is. This is an unofficial model answer, not an official UTM marking scheme.

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA